

252-0027

**Einführung in die Programmierung
Übungen**

Woche 6: References, Recursion, Precondition

**Joran Bühler
Departement Informatik
ETH Zürich**

Bonus

1. `boolean containsSubstringAt(String str, int position, String sub)`: Diese Methode soll überprüfen, ob `sub` im `String str` an der Position `position` vorkommt. Zum Beispiel:

- `containsSubstringAt("abcd", 0, "ab")`: `true`
- `containsSubstringAt("abcd", 1, "ab")`: `false`
- `containsSubstringAt("abcd", 4, "ab")`: `false`
- `containsSubstringAt("abcd", -1, "ab")`: `false`

Hinweis: Achten Sie darauf, dass die Methode auch `false` zurückgeben soll, wenn `position` kein valider Index des `Strings str` ist (also wenn `position` negativ oder zu gross ist).

2. `int countSubstrings(String str, String sub)`: Diese Methode soll zählen, wie oft der `String sub` als Substring in `str` vorkommt - die Substrings können sich hier überlappen. Zum Beispiel:

- `countSubstrings("aaa", "aa")`: `2`
- `countSubstrings("aaaa", "aa")`: `3`
- `countSubstrings("12341234123", "123412")`: `2`

Bonus

3. `int countDisjointSubstrings(String str, String sub)`: Diese Methode soll zählen, wie oft `sub` *ohne Überlappungen* in `str` vorkommt. Das erste Vorkommen (von links) eines Substrings `sub` trägt also zum Rückgabewert der Methode bei. Jedes weitere Vorkommen trägt nur zum Rückgabewert bei, wenn sich dieses nicht mit einem beitragenden Vorkommen weiter links in `str` überschneidet. Zum Beispiel:

- `countDisjointSubstrings("abdcdba", "abc")`: 1
- `countDisjointSubstrings("abdcdba", "c")`: 2
- `countDisjointSubstrings("0101010", "010")`: 2

My solution:

```
public static boolean containsSubstringAt(String str, int position, String sub) { 7 usages
    //base case: check for bounds
    if (position + sub.length() > str.length() || position >= str.length() || position < 0) {
        return false;
    }
    //return true if substring at position is equal
    return str.substring(position, position + sub.length()).equals(sub);
}
```

```
public static int countSubstrings(String str, String sub) { 4 usages
    int counter = 0;
    //check at each index if the substring is contained
    for(int i=0; i <= str.length()-sub.length(); i++){
        if(containsSubstringAt(str, i, sub)){
            //found substring at position i
            counter ++;
        }
    }
    return counter;
}
```

```
//should throw an error if substring.length() == 0
public static int countDisjointSubstrings(String str, String sub) { 4 usages
    int counter = 0;
    //note that I don't increase i in the for loop itself
    for(int i=0; i <= str.length()-sub.length();){
        if(containsSubstringAt(str, i, sub)){
            counter ++;
            i+=sub.length();
        }
        else i++;
    }
    return counter;
}
```

References

Value vs Reference

- Variablen enthalten entweder einen **Wert** (Value) oder eine **Referenz** (Reference).
- Variablen enthalten **nie** Objekte.
- Referenzen zeigen auf das Objekt im Speicher.

Value vs Reference - Beispiel

```
1 public class Example {  
2     public static void setX(int x) {  
3         x = 2;  
4     }  
5  
6     public static void main(String[] args) {  
7         int x = 3;  
8         setX(x);  
9         System.out.println(x);  3  
10    }  
11 }
```


Value vs Reference - Beispiel

```
1 public class Example {  
2     public static void setX(int x) {  
3         x = 2;  
4     }  
5  
6     public static void main(String[] args) {  
7         int x = 3;  
8         setX(x);  
9         System.out.println(x);  
10    }  
11 }
```

Scope "x2"

Scope "x1"

Value vs Reference

■ Referenzen folgen Value Semantics

- In Java speichern Variablen, die auf Objekte zeigen, keine Objekte selbst, sondern Referenzen (also Speicheradressen).
- Wenn man eine Referenz einer anderen zuweist, wird nur die Adresse kopiert, nicht das Objekt.
- Jede Referenz merkt sich also wo im Speicher das Objekt liegt.

■ Objekte selbst ermöglichen Reference Semantics

- Mehrere Referenzen können auf dasselbe Objekt zeigen.
- Änderungen über eine Referenz sind daher auch bei der anderen sichtbar.
- Eine Referenz ist also kein eigenständiges Objekt, sondern ein Zeiger auf ein Objekt im Speicher.

Value vs Reference - Beispiel

```
1  import java.util.Arrays;
2
3  public class Example {
4      public static void setX(int[] x) {
5          x = new int[] {4, 5, 6};
6      }
7
8  → public static void main(String[] args) {
9      int[] x = new int[] {1, 2, 3}; X
10     setX(x);
11     String xStr = Arrays.toString(x);
12     System.out.println(xStr);
13 }
14 }
```

[4,5,6]

[1,2,3]

Value vs Reference - Beispiel

```
1  import java.util.Arrays;
2
3  public class Example {
4      public static void setX(int[] x) {
5          x = new int[] {4, 5, 6};
6      }
7
8      public static void main(String[] args) {
9          → int[] x = new int[] {1, 2, 3};
10         setX(x);
11         String xStr = Arrays.toString(x);
12         System.out.println(xStr);
13     }
14 }
```

[4,5,6]

[1,2,3]

Value vs Reference - Beispiel

```
1  import java.util.Arrays;
2
3  public class Example {
4      public static void setX(int[] x) {
5          → x = new int[] {4, 5, 6};
6      }
7
8      public static void main(String[] args) {
9          int[] x = new int[] {1, 2, 3};
10         → setX(x);
11         String xStr = Arrays.toString(x);
12         System.out.println(xStr);
13     }
14 }
```

[4,5,6]

[1,2,3]

Value vs Reference - Beispiel

```
1 import java.util.Arrays;
```

```
2
```

```
3 public class Example {
```

```
4     public static void setX(int[] x) {
```

```
5          x = new int[] {4, 5, 6};
```

```
6     }
```

```
7
```

```
8     public static void main(String[] args) {
```

```
9         int[] x = new int[] {1, 2, 3};
```

```
10         setX(x);
```

```
11        String xStr = Arrays.toString(x);
```

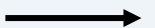
```
12        System.out.println(xStr);
```

```
13    }
```

```
14 }
```



[4,5,6]



[1,2,3]

Value vs Reference - Beispiel

```
1  import java.util.Arrays;
2
3  public class Example {
4      public static void setX(int[] x) {
5          x = new int[] {4, 5, 6};
6      }
7
8      public static void main(String[] args) {
9          int[] x = new int[] {1, 2, 3};
10         setX(x);
11         String xStr = Arrays.toString(x);
12         System.out.println(xStr);
13     }
14 }
```

Scope x2

Scope x1

[4,5,6]

[1,2,3]

“[1,2,3]”

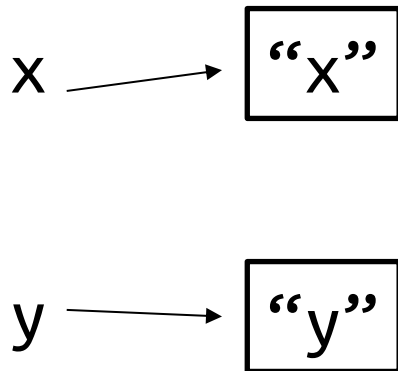
Value vs Reference - Beispiel

```
1  import java.util.Arrays;
2
3  public class Example {
4      public static void setX(int[] x) {
5          x[1] = 5;
6      }
7
8      public static void main(String[] args) {
9          int[] x = new int[] {1, 2, 3};
10         setX(x);
11         String xStr = Arrays.toString(x);
12         System.out.println(xStr);
13     }
14 }
```

→ “[1,5,3]”

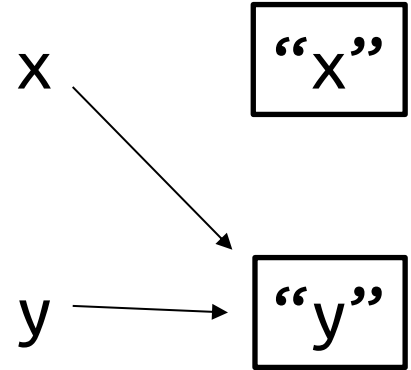
Value vs Reference - Beispiel

```
1 String x = "x";  
2 String y = "y";  
3  
4 //Assignment  
5 x = y;
```



Value vs Reference - Beispiel

```
1 String x = "x";  
2 String y = "y";  
3  
4 //Assignment  
5 x = y;
```



Value vs Reference - Beispiel

```
1  Int x = 5;  
2  Int y = 3;  
3  
4  // Assignment  
5  x = y;
```

X: 5

Y: 3

Value vs Reference - Beispiel

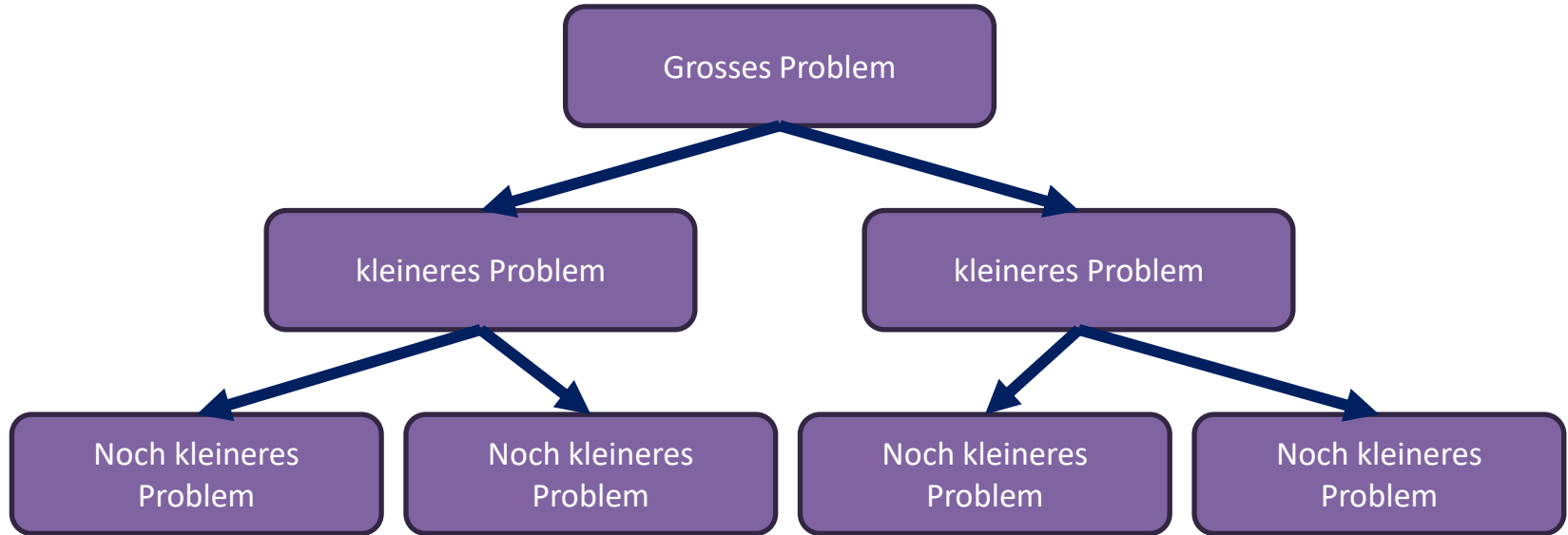
```
1  Int x = 5;  
2  Int y = 3;  
3  
4  // Assignment  
5  x = y;
```

X: 3

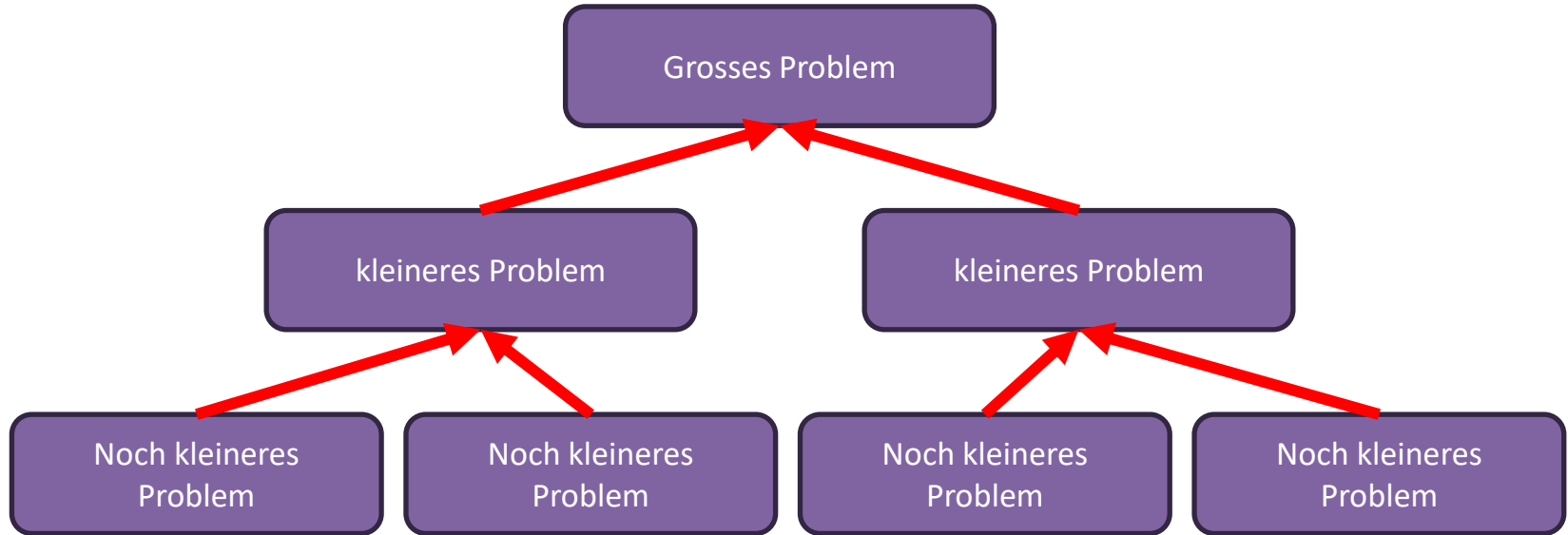
Y: 3

Rekursion

Rekursion – Grundprinzip Divide and Conquer



Rekursion – Grundprinzip Divide and Conquer



Decrease-and-Conquer - Fakultät

factorial(3)



```
public int factorial(int n) {  
    if (n==1) return 1;  
    return n*factorial(n-1);  
}
```


Decrease-and-Conquer - Fakultät

3*factorial(2)



```
public int factorial(int n) {  
    if (n==1) return 1;  
    return n*factorial(n-1);  
}
```

Decrease-and-Conquer - Fakultät



```
public int factorial(int n) {  
    if (n==1) return 1;  
    return n*factorial(n-1);  
}
```

3*factorial(2)



factorial(2)

Decrease-and-Conquer - Fakultät



```
public int factorial(int n) {  
    if (n==1) return 1;  
    return n*factorial(n-1);  
}
```

3*factorial(2)



2*factorial(1)

Decrease-and-Conquer - Fakultät



```
public int factorial(int n) {  
    if (n==1) return 1;  
    return n*factorial(n-1);  
}
```

3*factorial(2)



2*factorial(1)



factorial(1)

Base Case!

Decrease-and-Conquer - Fakultät



```
public int factorial(int n) {  
    if (n==1) return 1;  
    return n*factorial(n-1);  
}
```

3*factorial(2)



2*factorial(1)



1

Decrease-and-Conquer - Fakultät



```
public int factorial(int n) {  
    if (n==1) return 1;  
    return n*factorial(n-1);  
}
```

3*factorial(2)

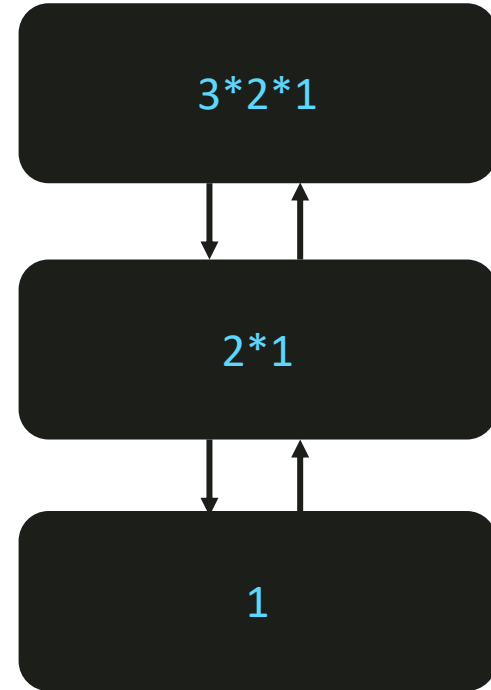
2*1

1

Decrease-and-Conquer - Fakultät



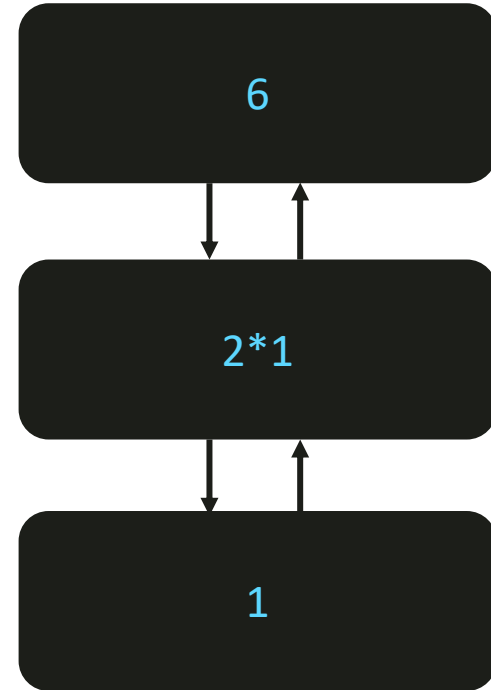
```
public int factorial(int n) {  
    if (n==1) return 1;  
    return n*factorial(n-1);  
}
```



Decrease-and-Conquer - Fakultät



```
public int factorial(int n) {  
    if (n==1) return 1;  
    return n*factorial(n-1);  
}
```



Rekursion – Step by Step

Problem, das wir lösen wollen

Aufgabe: Schreiben Sie eine Funktion $\text{sum}(n)$, die gegeben eine nichtnegative Zahl, alle nichtnegativen Zahlen bis einschliesslich n aufsummiert.

Beispiele:

$$\text{sum}(0) = 0$$

$$\text{sum}(1) = 1 = 1 + 0$$

$$\text{sum}(4) = 10 = 4 + 3 + 2 + 1 + 0$$

Wir wollen dies nun **rekursiv** lösen.

Schritte

1. Was ist die einfachste mögliche Eingabe?
2. Beispiele ausprobieren und visualisieren
3. Leite größere Beispiele von kleineren Beispielen ab
4. Verallgemeinere das Muster
5. Schreibe den Code

Schritte

1. Was ist die einfachste mögliche Eingabe?

→ Base Case

2. Beispiele ausprobieren und visualisieren

3. Leite größere Beispiele von kleineren Beispielen ab

4. Verallgemeinere das Muster

→ Rekursionsschritt

5. Schreibe den Code

1. Die einfachste Eingabe

- Hier ist es $n = _ : \text{sum}(n) = __$
- Die einfachste Eingabe ist später oft unser *Base Case*.
- Der Base Case ist der einzige Fall einer rekursiven Funktion, in dem wir eine direkte Lösung angeben.
- Eine Funktion kann auch mehrere Base Cases haben.
- Alle andere Lösungen bauen auf dem Base Case auf.

1. Die einfachste Eingabe

- Hier ist es **$n = 0$** : **$\text{sum}(0) = 0$**
- Die einfachste Eingabe ist später oft unser *Base Case*.
- Der Base Case ist der einzige Fall einer rekursiven Funktion, in dem wir eine direkte Lösung angeben.
- Eine Funktion kann auch mehrere Base Cases haben.
- Alle andere Lösungen bauen auf dem Base Case auf.

Schritte

1. Was ist die einfachste mögliche Eingabe?

→ Base Case

2. Beispiele ausprobieren und visualisieren

3. Leite größere Beispiele von kleineren Beispielen ab

4. Verallgemeinere das Muster

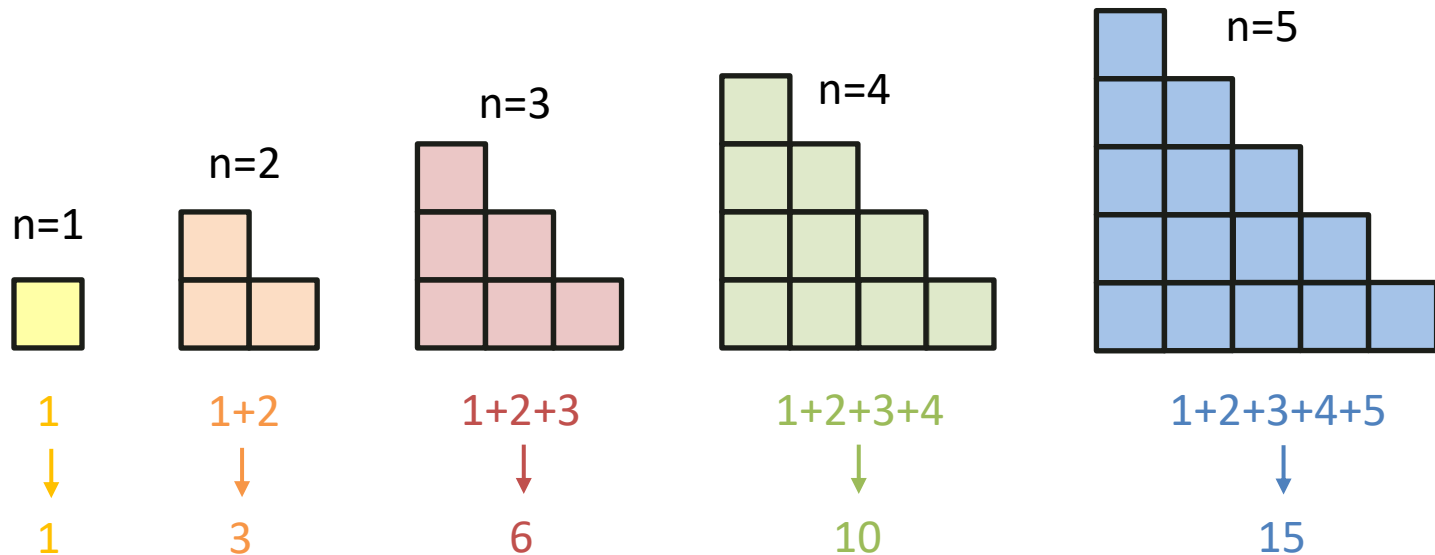
→ Rekursionsschritt

5. Schreibe den Code



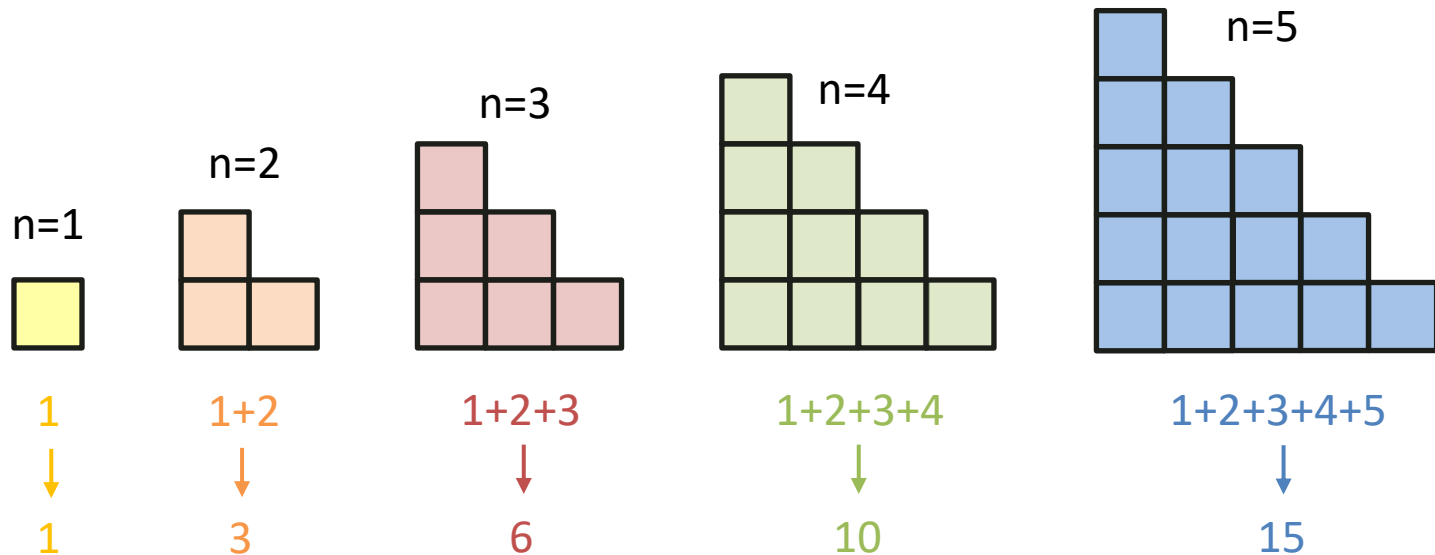
2. Die Beispiele

- Visualisiere, wie die Ein- und Ausgaben der Funktion miteinander zusammenhängen



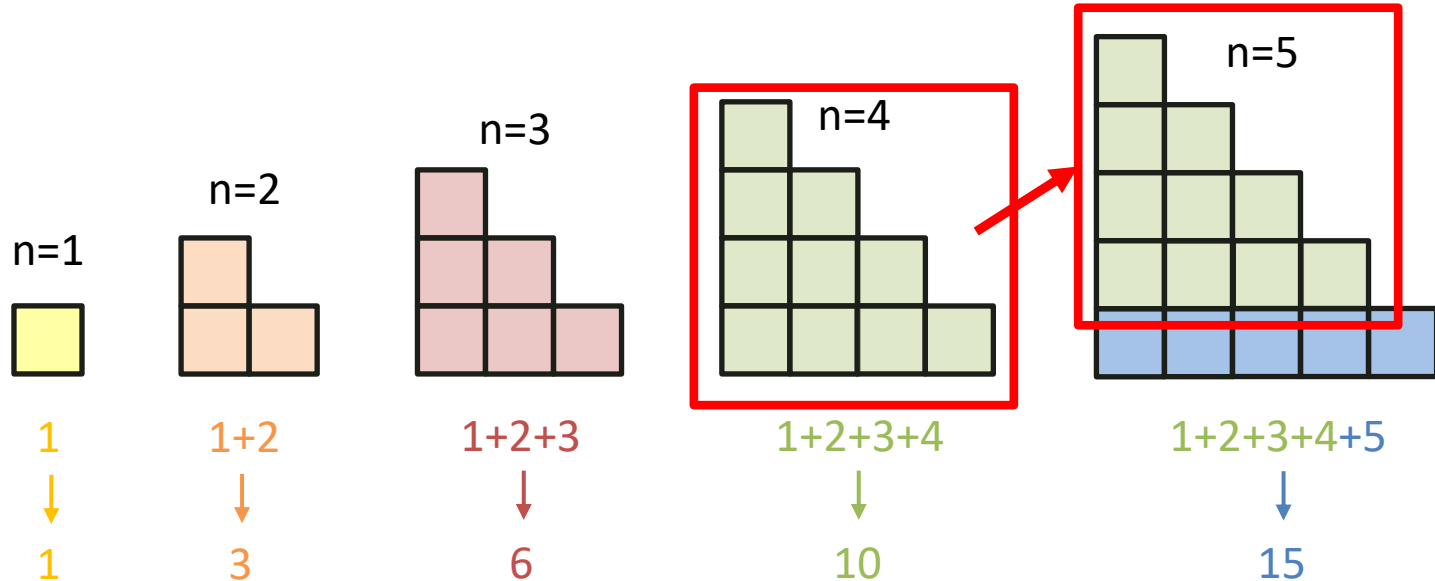
3. Erkenne den Zusammenhang

- „Wenn ich die Antwort für $n=4$ gegeben hätte, wie komme ich auf die Antwort für $n=5$? Für $n=3$ auf $n=4$? ...“



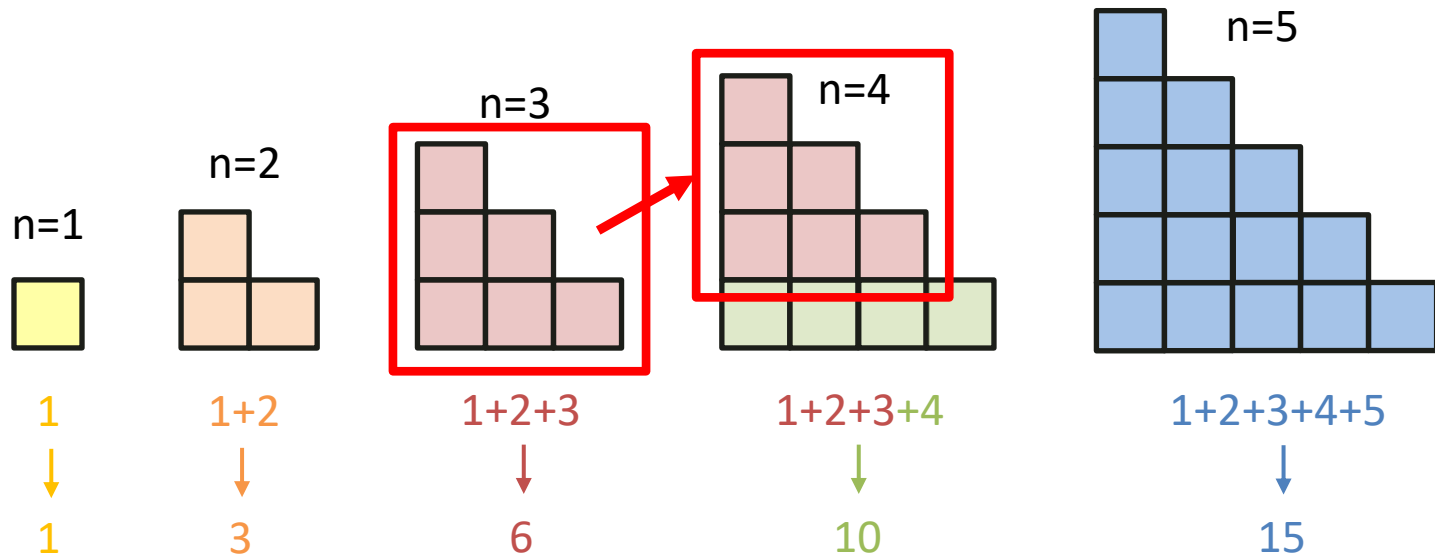
3. Erkenne den Zusammenhang

- „Wenn ich die Antwort für $n=4$ gegeben hätte, wie komme ich auf die Antwort für $n=5$? Für $n=3$ auf $n=4$? ...“



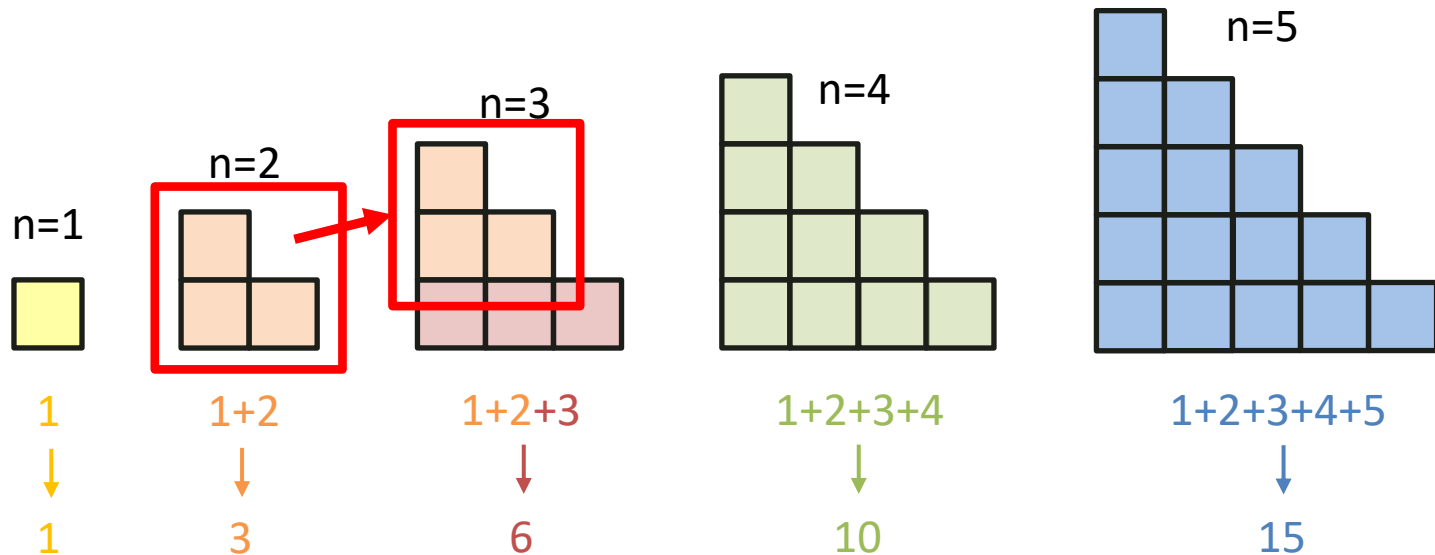
3. Erkenne den Zusammenhang

- „Wenn ich die Antwort für $n=4$ gegeben hätte, wie komme ich auf die Antwort für $n=5$? Für $n=3$ auf $n=4$? ...“



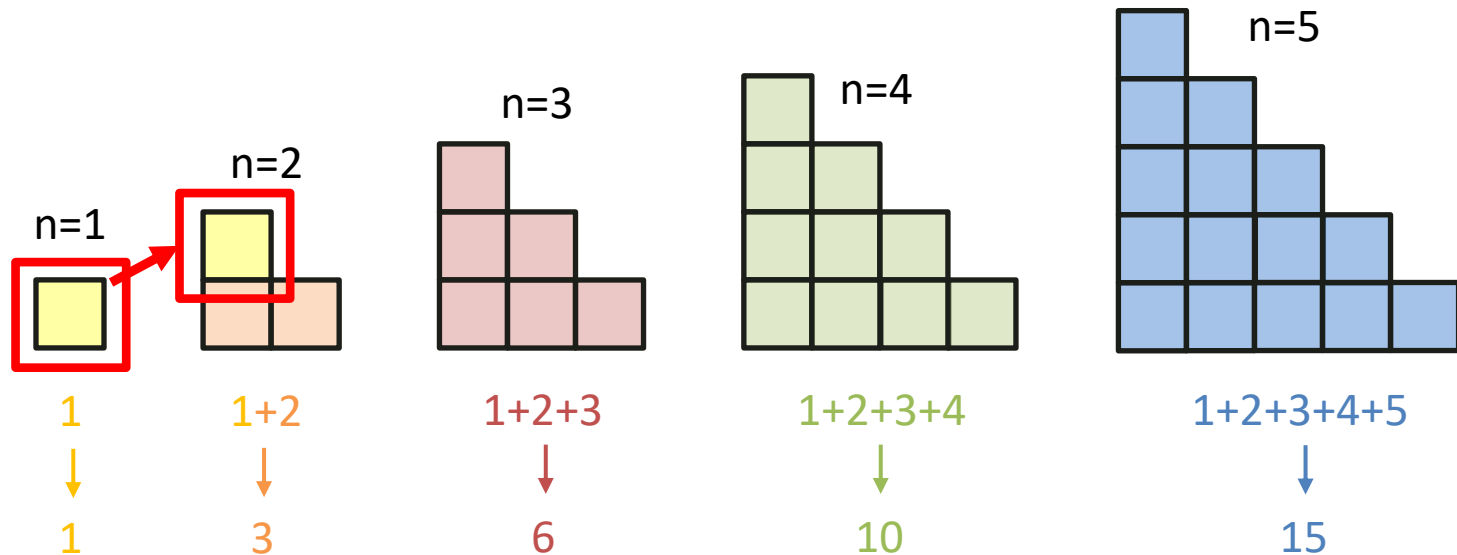
3. Erkenne den Zusammenhang

- „Wenn ich die Antwort für $n=4$ gegeben hätte, wie komme ich auf die Antwort für $n=5$? Für $n=3$ auf $n=4$? ...“

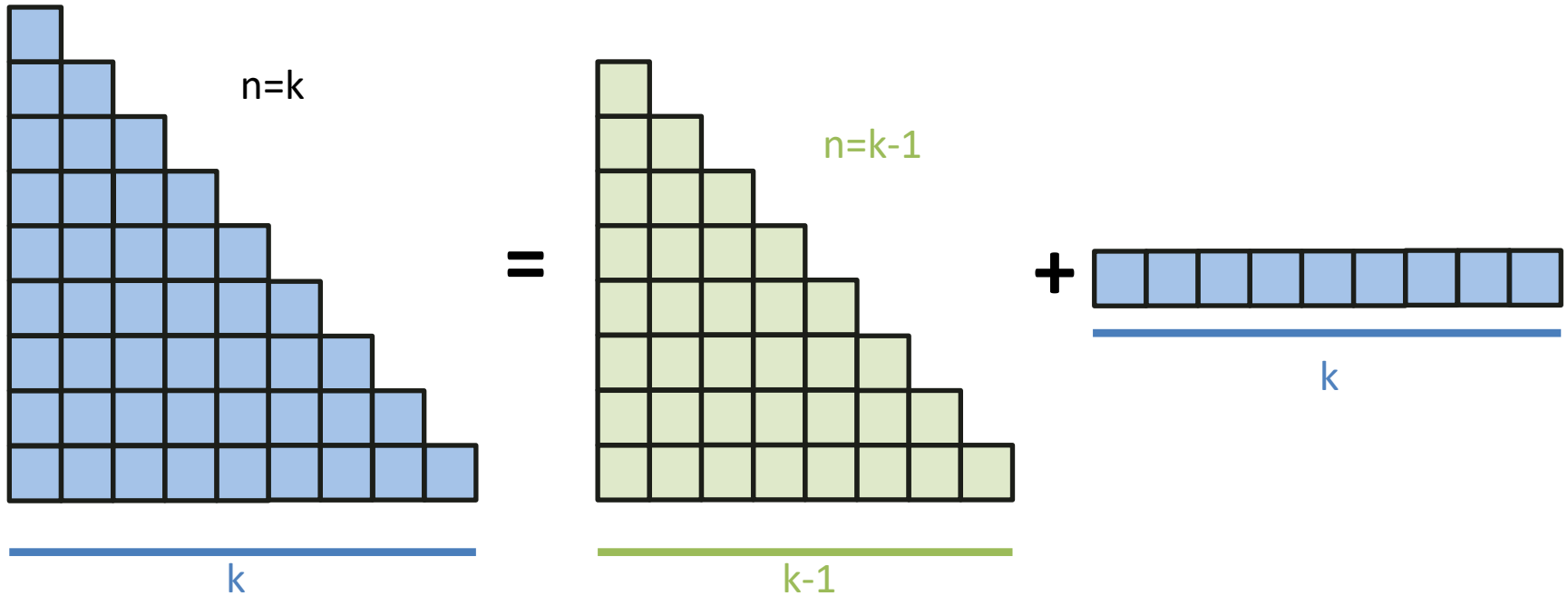


3. Erkenne den Zusammenhang

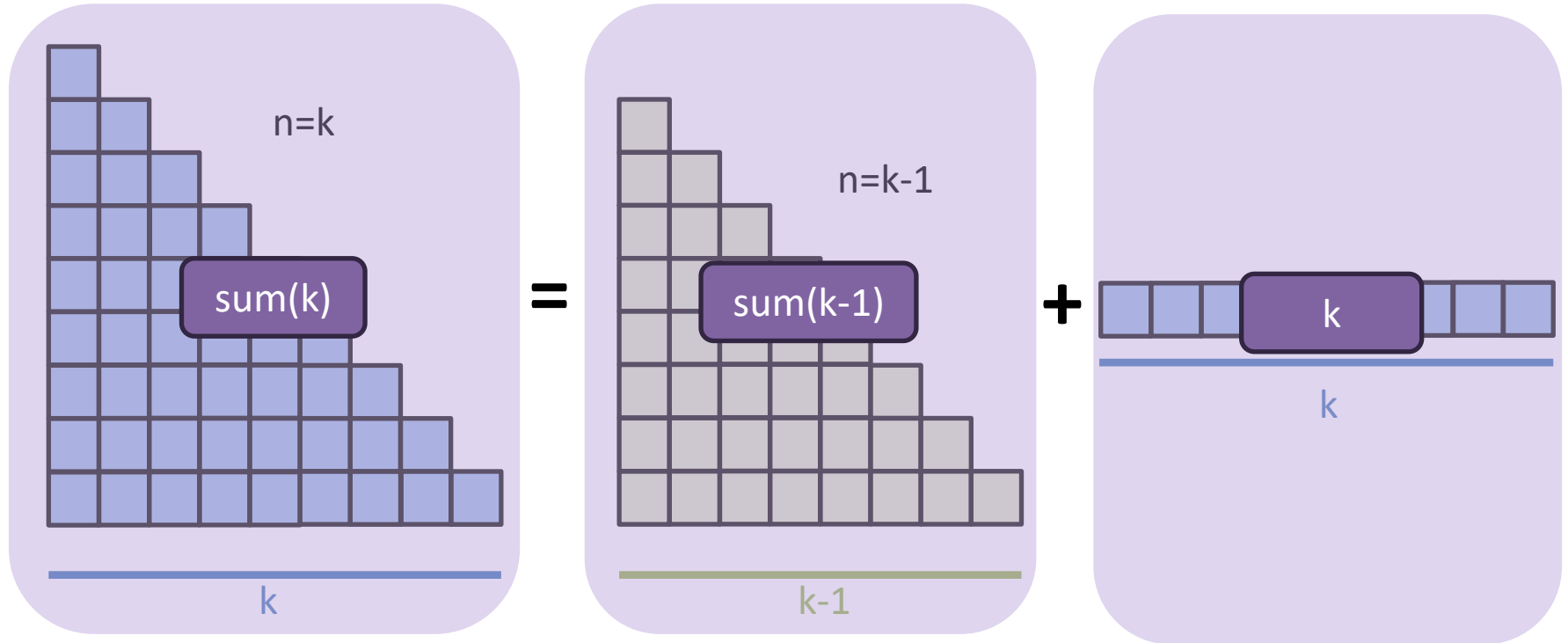
- „Wenn ich die Antwort für $n=4$ gegeben hätte, wie komme ich auf die Antwort für $n=5$? Für $n=3$ auf $n=4$? ...“



4. Das Muster



4. Das Muster



Schritte

1. Was ist die einfachste mögliche Eingabe?

→ Base Case



2. Beispiele ausprobieren und visualisieren

3. Leite größere Beispiele von kleineren Beispielen ab

4. Verallgemeinere das Muster

→ Rekursionsschritt



5. Schreibe den Code

5. Der Code

Was haben wir herausgefunden?

Im 1. Schritt haben wir den *Base Case* gefunden:

Falls $n=0$, dann $\text{sum}(n) = 0$

Danach haben wir das *Rekursionsmuster* gefunden:

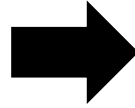
$\text{sum}(n) = \text{sum}(n-1) + n$

→ Dies können wir jetzt in Code umschreiben

5. Base Case + Muster $\rightarrow$ Code

Falls $n=0$: $\text{sum}(n) = 0$

Sonst: $\text{sum}(n) = \text{sum}(n-1) + n$



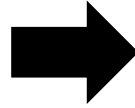
```
public static int sum(int n) {  
  
}  

```

5. Base Case + Muster $\rightarrow$ Code

Falls $n=0$: $\text{sum}(n) = 0$

Sonst: $\text{sum}(n) = \text{sum}(n-1) + n$

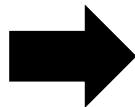


```
public static int sum(int n) {  
    if (n == 0) {  
        return 0;  
    }  
}
```

5. Base Case + Muster $\rightarrow$ Code

Falls $n=0$: $\text{sum}(n) = 0$

Sonst: $\text{sum}(n) = \text{sum}(n-1) + n$

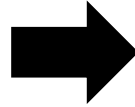


```
public static int sum(int n) {  
    if (n == 0) {  
        return 0;  
    }  
}
```

5. Base Case + Muster $\rightarrow$ Code

Falls $n=0$: $\text{sum}(n) = 0$

Sonst: $\text{sum}(n) = \text{sum}(n-1) + n$



```
public static int sum(int n) {  
    if (n == 0) {  
        return 0;  
    }  
    return sum(n-1) + n;  
}
```

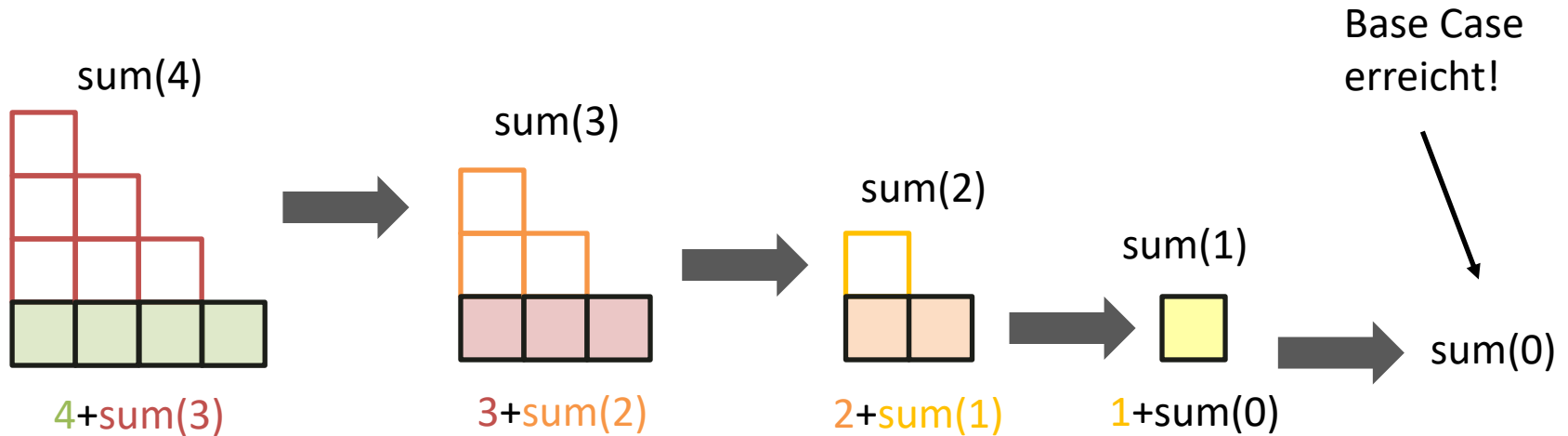
5. Base Case + Muster → Code

```
public static int sum(int n) {  
    if (n == 0) {  
        return 0;  
    }  
    return sum(n-1) + n;  
}
```

Fragen?

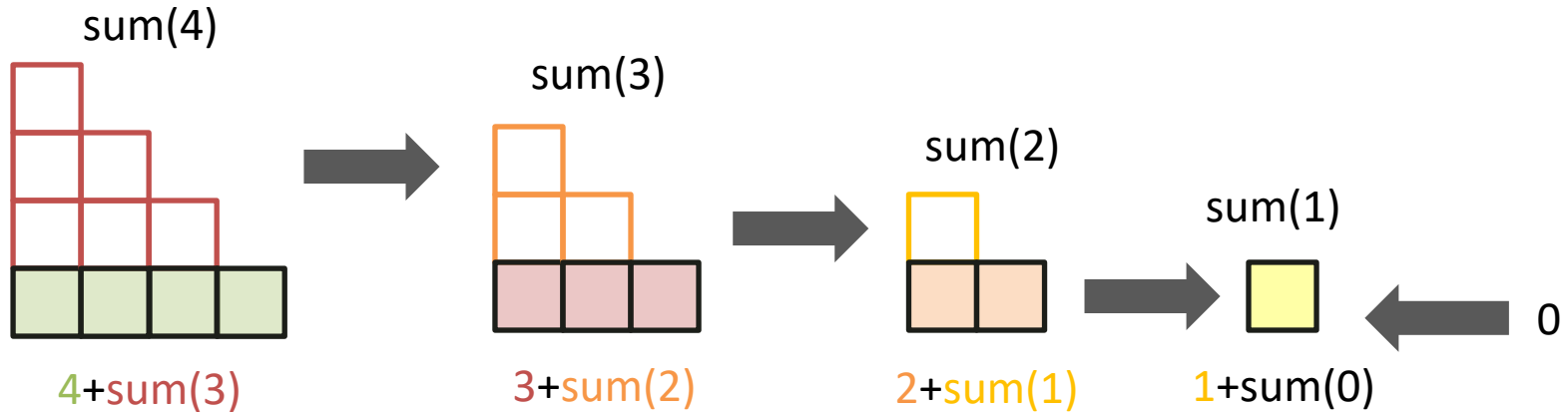
Was passiert wenn ich diesen Code ausführe?

1. Rekursive Aufrufe



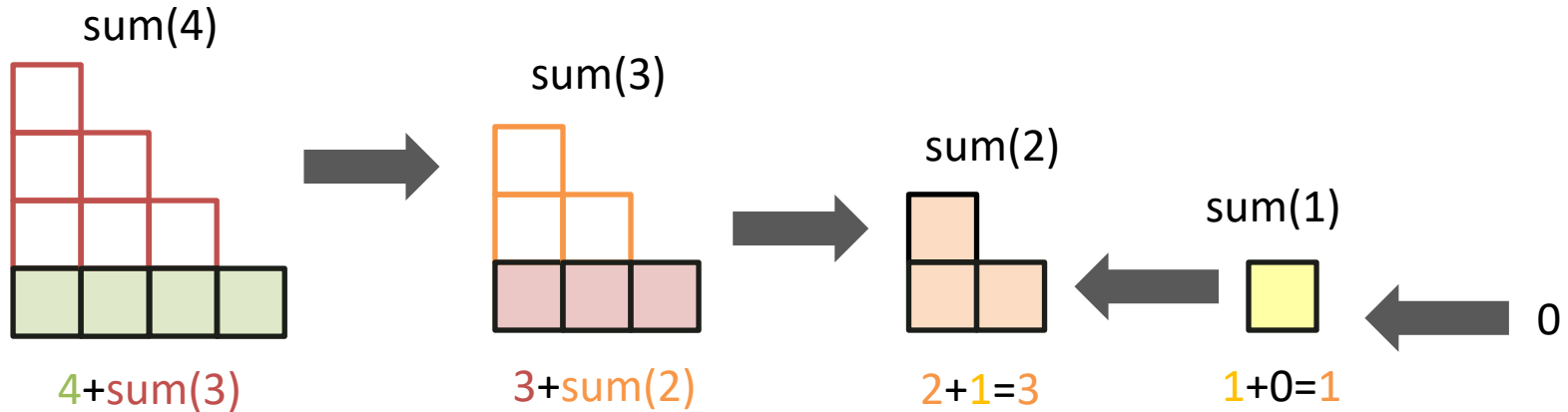
Was passiert wenn ich diesen Code ausführe?

2. Auflösen



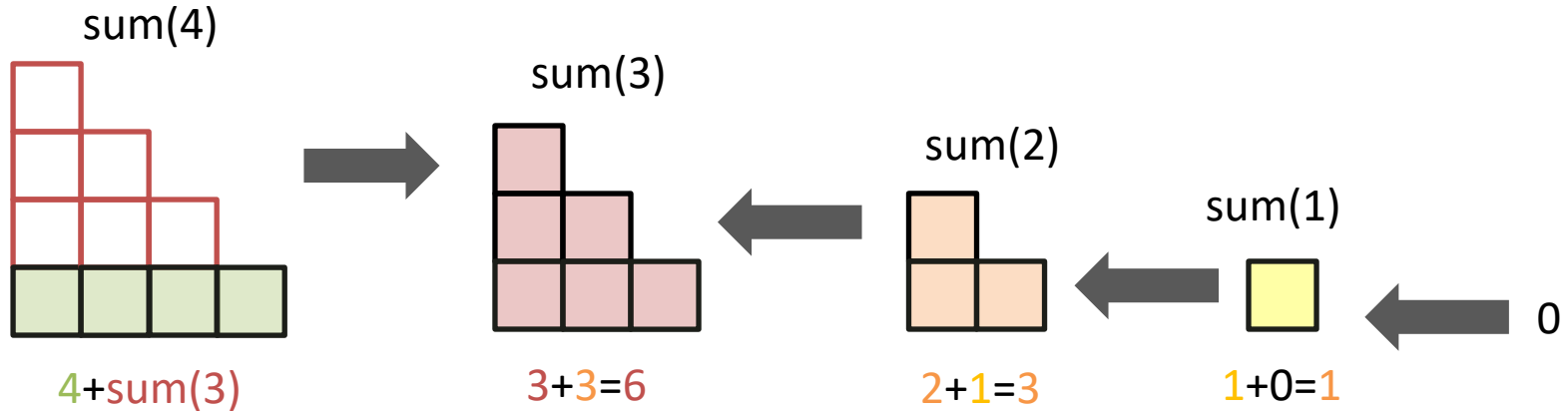
Was passiert wenn ich diesen Code ausführe?

2. Auflösen



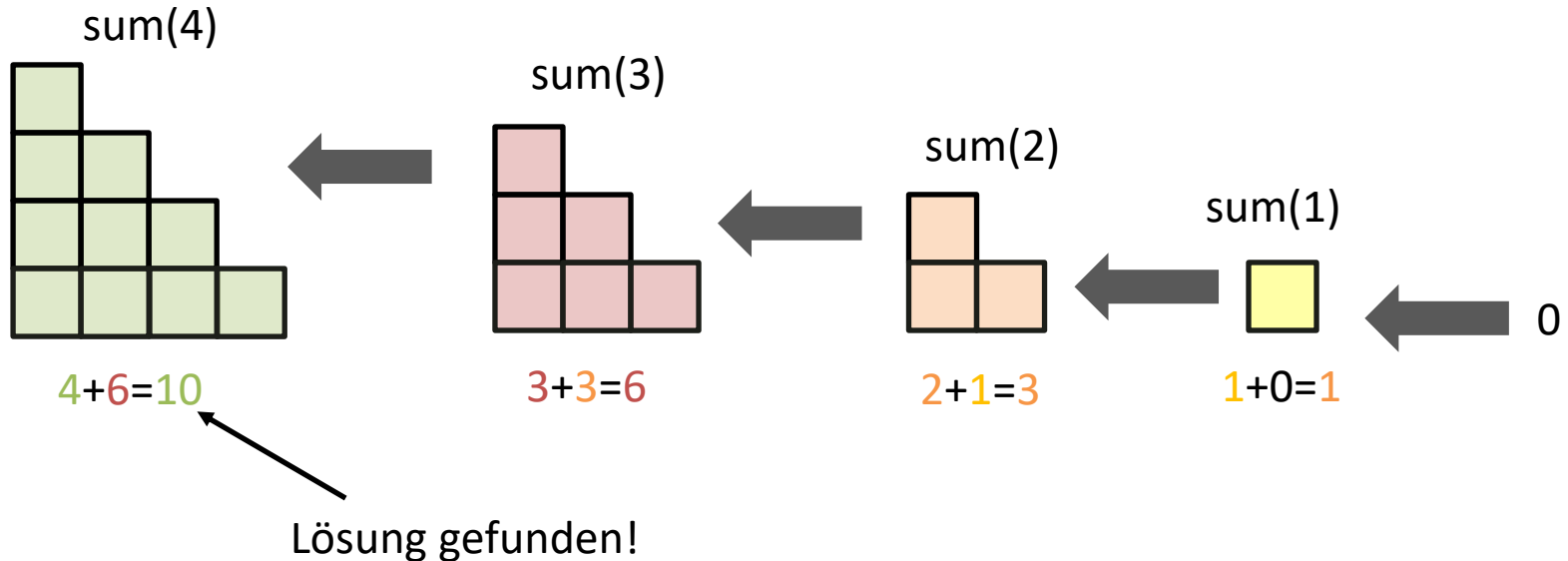
Was passiert wenn ich diesen Code ausführe?

2. Auflösen



Was passiert wenn ich diesen Code ausführe?

2. Auflösen



Rekursion in Java – Aufgabe

Erstelle eine Klasse `CheckIsPalindromRecursive`.

Implementiere eine **statische, rekursive Methode** `IsPalindrome`, die:

- Einen String akzeptiert
- Einen **boolean** zurückgibt, ob der gesamte String ein Palindrom ist.

Basisfall der Rekursion:

- Wenn die String-Länge 0 oder 1 beträgt, gib `true` zurück
- Bei einer Länge von 2: Prüfe, ob beide Zeichen gleich sind, und gib entsprechend `true` oder `false` zurück.

Rekursiver Fall:

- Nimm das **erste** und **letzte** Zeichen des Strings.
- Überprüfe, ob sie **gleich** sind.
- Verknüpfe das Ergebnis logisch mit einem Aufruf von `IsPalindrome` für den Teilstring ohne das erste und letzte Zeichen.

Tipp: Benutze `str.charAt(i)`, `str.substring(i, j)`

Java Debugger

Was ist der Debugger und was tut er?

- Ein Tool (in Java), das beim Debuggen hilft.
- Mit dem Java-Debugger kann man Schritt für Schritt durch das Programm gehen und genau beobachten, wie es ausgeführt wird.
- Die Änderungen der Variablen und ihrer Werte in jeder Zeile werden in einer Tabelle dargestellt.

Hoare Logik

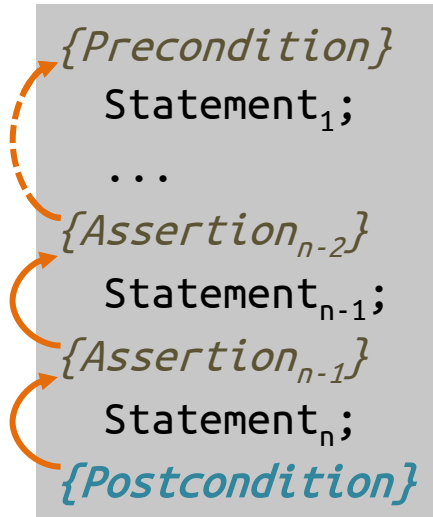
Hoare-Tripel: Syntax

- Ein **Hoare-Tripel** («**Hoare triple**»), auch *3-Tupel* genannt, besteht aus zwei Aussagen und einem Programmsegment:

$$\{P\} \ S \ \{Q\}$$

- Bestandteile eines Tripels
 - P ist die Vorbedingung (Precondition, z.B. $u > -59$)
 - S ist eine Anweisung (Statement; bzw. mehrere Anweisungen)
 - Q ist die Nachbedingung (Postcondition, z.B. $z > -59$)

Rückwärtsschliessen: Vorgehen



- **Start:** Wählen (wissen, raten) einer sinnvollen *Nachbedingung*
 - **Schrittweise:** Herleiten der vorherigen Aussage (*Assertion_{i-1}*) durch Einbeziehen des *Effekts* der vorherigen Anweisung (*Statement_i*)
 - **Ziel:** Herleiten einer *notwendigen und hinreichenden Vorbedingung*
-
- Rückwärts = «Welche Vorbedingung braucht mein Code, damit er die gewählten Garantien (Nachbedingung) geben kann?»

Typische Aufgaben:

- **Ist ein Hoare-Tripel gültig oder ungültig?**
 - Vor- und Nachbedingung gegeben
 - Folgt die Nachbedingung aus dem Code und der Vorbedingung?
- **Was ist die schwächste Precondition?**
 - Postcondition ist gegeben
 - Welche Werte erfüllen unsere Postcondition?

Stärkere und schwächere Aussagen

- Wir können *stärkere* und *schwächere* Aussagen unterscheiden
 - Wenn $P_1 \Rightarrow P_2$ gilt, dann ist P_1 stärker als P_2 (und P_2 schwächer als P_1)
 - Falls P_1 wahr ist, ist P_2 auch wahr.
 - Bsp: $P_1: x > 4$ (Stärker) $P_2: x > 0$ (Schwächer)
- **Starke Aussage:** Eine Aussage oder Voraussetzung, die **mehr Einschränkungen** auferlegt. Sie gilt in weniger Fällen.
- **Schwache Aussage:** Eine Aussage oder Voraussetzung, die **weniger Einschränkungen** auferlegt. Sie ist allgemeiner.

Preconditions / Postconditions

- Die **schwächste Precondition** ist die schwächste Precondition, für die noch die Postcondition erfüllt ist.
 - «Was dürfen wir der Funktion für Werte geben, damit wir einen erwünschten Output haben?»
 - Wir wollen viele mögliche Werte zulassen
- Die **stärkste Postcondition** ist die stärkste Postcondition, welche noch aus der Precondition folgt.
 - «Was kann alles als Resultat kommen?»
 - Wir wollen einschränken, was für Werte wir bekommen können

Weakest Precondition – Einfaches Beispiel

$$\{ \hspace{10cm} \}$$
$$X = Y^* Y \quad \underline{\hspace{1.5cm}}$$
$$\{x > 4\}$$

Weakest Precondition – Einfaches Beispiel

{ }

$x = y * y$ ————— $\{x > 4\}$

Weakest Precondition – Einfaches Beispiel

$$\{y * y > 4\}$$

$$x = y * y \text{ —————}$$

Weakest Precondition – Einfaches Beispiel

$$\{|y| > 2\}$$

$$x = y * y \text{ —————}$$

Weakest Precondition – Zweites Beispiel

{ }

$y = x + 3$

$z = y + 1$

$\{z > 4\}$

Weakest Precondition – Zweites Beispiel

{ }

$y = x + 3$ —————
 $z = y + 1$ ————— $\{z > 4\}$

Weakest Precondition – Zweites Beispiel

{ }

$y = x + 3$ —————
 $z = y + 1$ ————— $\{y + 1 > 4\}$

Weakest Precondition – Zweites Beispiel

{ }

$y = x + 3$ $\{y + 1 > 4\}$

$z = y + 1$ _____

Weakest Precondition – Zweites Beispiel

{ }

$y = x + 3$ $\{x + 3 + 1 > 4\}$

$z = y + 1$ _____

Weakest Precondition – Zweites Beispiel

{ }

$y = x + 3$ ————— $\{x > 0\}$

$z = y + 1$ —————

Weakest Precondition – Zweites Beispiel

$$\{x > 0\}$$

$$y = x + 3 \quad \text{—————}$$

$$z = y + 1 \quad \text{—————}$$

Schwächste Vorbedingung - Beispiel

{ }

$x = a - 4;$

$\{x > 0\}$

Schwächste Vorbedingung - Beispiel

{ }

$x = y + z;$

$y = y - 5;$

$\{y < 0\}$

Kahoot

FS 21 Prüfungsaufgabe

Bitte geben Sie für die folgenden Programmsegmente die schwächste Vorbedingung (weakest precondition) an. Bitte verwenden Sie Java Syntax (die Klammern { und } können Sie weglassen). Alle Anweisungen sind Teil einer Java Methode. Alle Variablen sind vom Typ int und es gibt keinen Overflow.

1. P: ??

```
m = n * 4; k = m - 2;  
Q: { n > 0 && k > 5 }
```

2. P: ??

```
if (x > y) {  
    z = x - y;  
} else {  
    z = y - x;  
}  
{ z > 0 }
```

$$n > 1$$

$$\begin{aligned} n > 0 \text{ \&\& } n \cdot 4 - 2 > 5 &\Rightarrow \underline{n \cdot 4 > 7} \\ n > 0 \text{ \&\& } (n - 2) > 5 \end{aligned}$$

$$\begin{aligned} &\underline{(x - y > 0 \text{ \&\& } x > y)} \parallel y - x > 0 \\ &\text{\&\& } x \leq y \end{aligned}$$

Bonusaufgabe

Aufgabe 6: Matrix (Bonus!)

Achtung: Diese Aufgabe gibt Bonuspunkte (siehe “Leistungskontrolle” im www.vvz.ethz.ch). Die Aufgabe muss eigenhändig und alleine gelöst werden. Die Abgabe erfolgt wie gewohnt per Push in Ihr Git-Repository auf dem ETH-Server. Verbindlich ist der letzte Push vor dem Abgabetermin. Auch wenn Sie vor der Deadline committen, aber nach der Deadline pushen, gilt dies als eine zu späte Abgabe. Bitte lesen Sie zusätzlich [die allgemeinen Regeln](#).

Implementieren Sie die Methode `countAssimilated` in der Klasse `Matrix`, welche eine $m \times n$ -Matrix A (mit $m > 2, n > 2$) von `int`-Werten als Input akzeptiert und einen `int`-Wert zurückgibt. Für alle Elemente $a_{i,j}$ von A gilt $a_{i,j} \neq 0$.

Diese Methode soll die Anzahl der *an ihre Umgebung angepassten* Elemente in der Input-Matrix A berechnen und zurückgeben. Ein Element $a_{i,j}$ von A gilt als angepasst, wenn die Summe seiner direkten Nachbarn (vertikal, horizontal und diagonal, soweit in der Matrix A definiert) ohne Rest durch $a_{i,j}$ teilbar ist. Das heisst, dass ein Element $a_{i,j}$ genau dann angepasst ist, wenn

$$(a_{i-1,j-1} + a_{i-1,j} + a_{i-1,j+1} + a_{i,j-1} + a_{i,j+1} + a_{i+1,j-1} + a_{i+1,j} + a_{i+1,j+1}) \% a_{i,j} == 0$$

gilt. Wenn das Element $a_{x,y}$ nicht existiert (da $x < 0$ oder $x \geq m, y < 0, y \geq n$), dann wird der Wert in der Berechnung durch 0 ersetzt.

Sie können davon ausgehen, dass der Parameter `matrix` nie den Wert `null` hat und nie auf eine $m \times n$ Matrix mit $m \leq 2$ oder $n \leq 2$ verweist. Sie können auch davon ausgehen, dass die Summe der Nachbarn sich ohne Overflow oder Underflow berechnen lässt.